
SmartShelfKart

System architecture and technical reference

DOCUMENT	Engineering reference, revision 1
SCOPE	Data model, authorization, domain modules, inventory mathematics, the assistant service, and the delivery pipeline
AUDIENCE	Engineers joining the codebase, and reviewers assessing it
STATUS	Describes the system as built. Sections 13 and 14 record open items and the reasoning behind decisions that look unusual.

Contents

1. What the system is

Runtimes and where work happens · Why the assistant is a separate service

2. Data model

Tenancy · Collections · The transaction ledger · Indexes

3. Authorization

Two enforcement points, and why · Membership against permission · Cache lifetimes · Platform administration

4. Domain modules

5. Inventory mathematics

Demand · Safety stock and the reorder point · Replenishment quantity · Classification

6. The assistant

Request path · The fact layer · The deterministic bank · Routing and cost tiers · Product resolution · Writes: preview, confirm, execute · Bulk actions · Guardrails and the autonomous swarm · Answer caching · Why there is no vector search

7. The financial read model

Money · Tenancy is the physical layout · The reports · Authorization, for the third time · The sync is a full rebuild · Portable SQL

8. Grading the assistant

What the golden set asserts · Ground truth · Scripting the model, not the system · Deterministic coverage as a cost gate · Testing the graders

9. Observability

Spans · Metrics · Privacy

10. The Model Context Protocol surface

11. The API contract

12. Delivery

Pipeline · Two traps in the test setup

13. Deployment

14. Decisions and open items

1 What the system is

SmartShelfKart is a multi-tenant inventory and operations platform. A tenant — called a *company* or *workspace* throughout the code — holds a product catalog, a ledger of every stock movement against it, and roughly thirty business modules built on top of those two things: purchasing, sales, warehousing, light manufacturing, service jobs, and the financial reporting that falls out of them.

Layered over that is a natural-language assistant that answers questions about live inventory and, on confirmation, changes it. The assistant is the part of the system with the most unusual engineering in it, and sections 6 through 10 are mostly about it.

1.1 Runtimes and where work happens

Four runtimes, with a deliberate imbalance in how much each is trusted.

RUNTIME	RESPONSIBILITY	TRUST
Client application	All user interaction; reads and writes Firestore directly for ordinary operations	None. Every write it makes is re-checked by security rules.
Firestore and Cloud Functions	System of record. Rules are the primary authorization boundary. One function reacts to user registration.	Rules are authoritative for anything reaching Firestore over the client SDK.
Assistant service (Python)	Natural-language question answering and agent-driven writes, over HTTP and over the Model Context Protocol	Runs with Admin SDK credentials, which bypass rules entirely — so it re-implements the authorization model itself. See section 3.
Reporting service (Java)	Financial reports over a PostgreSQL projection of the operational data: tax, aging, profit and loss, customer exposure	Also Admin SDK, so also re-implements the model. Read-only with respect to business data — it writes nothing back. See section 7.

The client talks to Firestore directly rather than through an API tier. For a system whose core operations are document reads and writes with per-field authorization, an intermediate service would mostly have been a place to re-implement the rules. The exception is the assistant, which needs to run a model, hold conversation state, and execute multi-step tool calls — none of which belongs on a device.

1.2 Why the assistant is a separate service

Three reasons, in descending order of how much they mattered:

1. **The model call cannot be made from the client.** Shipping an API key in an application bundle is shipping the key. Every request has to pass through something that holds the credential and can refuse to spend it.
2. **Answering well requires derived data the client does not have.** The assistant reasons over burn rates, demand variance and days of cover computed from a rolling 90-day window of the transaction ledger. Computing that on device, per question, over a catalog of any size, is not viable.
3. **Writes need a choke point.** A conversation that can change stock needs one place where permission is checked, one place where a preview is turned into a committed movement, and one place where a refusal is final. Section 6.6 covers it.

2 Data model

2.1 Tenancy

Almost everything lives under `companies/{companyId}/`. Tenancy is structural rather than a filter applied at query time, which means the isolation argument is short: a document either sits under a company path the caller has proved membership of, or the rules never evaluate it. There is no tenant column that a forgotten `where` clause can leak past.

Two things sit outside that tree. `users/{userId}` holds the account and its current workspace binding. `superAdmins/{uid}` and `platformAuditLogs/` belong to the platform operator rather than to any tenant. A small number of read-only documents — `publicConfig/`, `plans/` — are shared and world-readable by design, because the marketing site renders pricing from them.

NOTE — there is no default workspace, anywhere, in any layer. The assistant service raises rather than falling back when no company id is supplied, and the Model Context Protocol server refuses to start answering without one. An earlier version defaulted to a sandbox tenant, and defaulting is precisely how one workspace ends up reading another's stock.

2.2 Collections

Forty-odd subcollections hang off a company. Grouped by what they are for:

AREA	COLLECTIONS
Catalog	<code>products</code> , <code>categories</code> , <code>batches</code> , <code>serials</code> , <code>priceHistory</code> , <code>priceLists</code>
Movement	<code>transactions</code> , <code>stockTakes</code> , <code>stockHolds</code> , <code>transferOrders</code> , <code>warehouseZones</code>
Purchasing	<code>purchaseOrders</code> , <code>vendors</code> , <code>requisitions</code> , <code>landedCosts</code>
Sales	<code>salesOrders</code> , <code>quotations</code> , <code>invoices</code> , <code>recurringInvoices</code> , <code>customers</code> , <code>shipments</code> , <code>returns</code> , <code>billingSequences</code>
Production and service	<code>boms</code> , <code>jobWorkOrders</code> , <code>serviceJobs</code>
Money	<code>expenses</code> , <code>budgets</code> , <code>commissionPlans</code> , <code>registerSessions</code>
Governance	<code>members</code> , <code>roles</code> , <code>auditLogs</code> , <code>invites</code> , <code>joinCodeIndex</code> , <code>notifications</code> , <code>feedback</code> , <code>metadata</code>

2.3 The transaction ledger

`transactions` is the collection everything analytical is derived from, and it is the one worth understanding first. A product document carries a current quantity, but that number is a cache — the ledger is what actually happened. Burn rate, demand variance, days of cover, dead-stock age and available-to-promise are all computed from the ledger, never from the product document.

This has a consequence the assistant is careful about: a workspace with products but no recorded movements cannot have a burn rate. Not zero — undefined. The fact layer exposes `history_is_reliable` for exactly this, and every surface that would otherwise report "0 units per day" is required to say instead that demand is not tracked. Reporting zero would be a fabricated number dressed as a measurement.

2.4 Indexes

`firestore.indexes.json` defines the composite indexes the listing screens and the fact layer's windowed transaction reads need. It is 258 lines and worth reading before adding a query with more than one range or ordering clause, because Firestore will accept the query in the emulator and fail it in production.

3 Authorization

This is the part of the system most worth reviewing carefully, because it is enforced twice, in two languages, and the second enforcement exists only because the first one can be bypassed.

3.1 Two enforcement points, and why

`firestore.rules` — 980 lines — governs everything the client does. It is the primary boundary and it is where the model is defined: seventeen helper functions resolving membership, role grants, per-user overrides, company status and platform administration.

The assistant service does not go through it. It writes with the Firebase Admin SDK, and the Admin SDK ignores security rules completely. Whatever the rules withhold has to be withheld again in Python, or the assistant becomes a way around every permission in the product. Concretely: a user denied `canStockIn` in the application could otherwise ask the assistant to add five hundred units and it would succeed.

So `auth.py` re-implements the resolution — admin and owner short-circuit, then `companies/{cid}/roles/{roleId}`, then per-user overrides — and `nodes.may_run_tool` applies it at the single point every chat-driven write passes through. A refusal there cannot be routed around by rewording the request.

3.2 Membership against permission

These are separate checks and conflating them was a real bug once. Membership answers "does this caller belong to this workspace at all", and is proved by a server-side document at `companies/{cid}/members/{uid}` — never by a header. An earlier version trusted the `x-company-id` header alone, which meant any client could read any tenant's inventory by changing a string.

Permission answers "may this caller do this particular thing", and is required on top of membership for every mutating route. There are about 120 permission keys, and they are the same strings on both sides of the system, deliberately, so that the mapping between an assistant tool and an application capability is a lookup rather than a translation:

```
WRITE_TOOL_PERMISSIONS = {
    "create_product":      "canAddProducts",
    "update_stock":        "canAdjustStock",
    "create_purchase_order": "canCreatePurchaseOrders",
    "transfer_stock":      "canTransfer",
    "audit_inventory":     "canAdjustStock",
    "set_reorder_threshold": "canEditProducts",
    "bulk_action":         "canAdjustStock",
}
```

There is a third caller state besides "has the grant" and "does not": grants that could not be read at all, because Firestore was unavailable. The graph receives `None` for that case and refuses the write. A storage hiccup denies rather than permits, and there is a test that holds that line.

FIXED IN THIS REVISION — `may_run_tool` read an empty permission mapping as "this tool needs no permission" and returned true. The bulk dispatcher had no mapping, and although bulk requests are checked against their inner tool in practice, the fallback in that lookup resolved to the dispatcher sentinel, which mapped to nothing. A malformed bulk plan would have needed no permission at all. Both the sentinel and any unmapped write tool now fail closed.

3.3 Cache lifetimes

Membership is cached for 300 seconds; permissions for 30. The asymmetry is intentional and the reasoning is worth keeping: a stale *membership* record delays someone's removal from a workspace, which is bad but bounded. A stale *permission* record keeps granting a capability that was just revoked, which is the thing you revoked it to stop. So the more dangerous cache is the shorter one.

3.4 Platform administration

A separate super-admin tier operates across tenants — plan catalog, pricing, company status, platform audit log. It is gated by `superAdmins/{uid}` rather than by any role inside a company, so no amount of privilege inside a workspace reaches it, and its actions are written to `platformAuditLogs` rather than to the tenant's own audit trail.

4 Domain modules

What follows is the full set, with the constraint each one actually enforces. The common thread is that stock only ever moves through a path that writes a ledger entry — there is no module that adjusts a product quantity directly, because a quantity that changed without a transaction behind it makes every derived figure wrong from that moment on.

4.1 Catalog and stock

MODULE	WHAT IT DOES AND WHAT IT GUARANTEES
Products and categories	SKU and barcode tracking, per-product low-stock thresholds, cost and selling price, multi-location quantities.
Batches	Expiry-dated lots beneath a product, so a recall or an expiry sweep can name the affected units rather than the whole SKU.
Serial numbers	Individual units below the batch, with status, location, warranty dates and full movement history. Lookup is case-insensitive, which matters when a serial is read off a label by hand.
Stock takes	Blind counts reconciled against system quantity; the variance posts as a transaction, so a correction is visible in the ledger rather than silently overwriting a number.
Stock holds	Reserves units against a commitment without moving them, and is what makes available-to-promise differ from on-hand.
Warehouse zones	Named locations within a site. A product holding stock in more than one location must name the location on any movement.
Price lists and price history	Per-customer prices, blanket discounts and quantity slabs, applied in both invoicing and the till. Every cost-price change is written to history, which is what makes landed-cost reversal exact.
Label printing	Printable label sheets with Code 128, EAN-13 or QR in configurable grids.

4.2 Purchasing

MODULE	WHAT IT DOES AND WHAT IT GUARANTEES
Purchase orders	Draft through sent, received and closed, with partial receipts. Receiving is the only path that increases stock from a supplier.
Requisitions	Requests stock without committing company money. Approval is its own permission, and an approved request converts to a draft purchase order rather than to stock.
Landed costs	Spreads freight, duty and handling across a receipt by value, quantity or evenly. Applying updates cost prices and writes price history; reversal restores the previous values exactly, which is only possible because of that history.
Vendor scorecard	On-time delivery, fill rate, lead time against promise and price movement, graded A–D. Computed entirely from purchase orders already in the workspace, so it needs no separate data entry and cannot drift from reality.

4.3 Sales

MODULE	WHAT IT DOES AND WHAT IT GUARANTEES
Quotations	Priced offers with a validity window. An accepted quote converts to a sales order at the prices the customer agreed to, not at current prices.
Sales orders	Confirmation, dispatch and delivery. Dispatch through the sales order is the only path that removes stock for a sale.
Pick, pack and ship	Ordered, picked and packed tracked per line, so a short pick is caught at the bench rather than at the customer.
Invoices and recurring invoices	Numbered from <code>billingSequences</code> . Recurring generation is idempotent — the guard exists because the failure mode is billing a customer twice.
Returns	Customer and vendor returns, with approval and rejection as distinct permissions.
Credit control	Credit limits, payment terms and holds per customer, with live exposure computed from unpaid invoices. Checked at the invoice screen and at the till — before credit is extended, rather than after.
Fast POS and register sessions	Open a till with a float, record drops and payouts, close on a blind count. Every sale is stamped with its shift, so the closing report reconciles against what should be in the drawer.
Commissions	Revenue or margin basis, per-category rates, payable on collection rather than on issue. Statements are computed from invoices so anyone can reproduce a figure.

4.4 Movement, production and service

MODULE	WHAT IT DOES AND WHAT IT GUARANTEES
Transfer orders	Draft, dispatched, received, with partial receipts and shortage reporting. Stock in transit sits in an explicit bucket, so totals reconcile at every point of the journey rather than only at both ends.
Bills of materials	A finished product defined as components with per-component wastage.
Assembly	Build and unbuild in one atomic movement — components out, finished goods in — so an interrupted build cannot leave the ledger half-done.
Job work	Issue material to a subcontractor and receive finished goods back, with the outstanding balance tracked in between.

Service jobs	Book a unit in against its serial, diagnose, fit parts, hand it back. Parts leave stock in the same transaction, so a service department cannot quietly lose spares.
--------------	--

4.5 Financial and governance

MODULE	WHAT IT DOES AND WHAT IT GUARANTEES
Expenses	Rent, wages, freight out and the rest, by head and month. This is what lets the profit-and-loss report state a net profit rather than a gross margin.
Budgets and variance	Period budgets against actuals, pace-aware: 60% spent is fine in month eight and a problem in month two, and the report says which.
Tax summary	Output and input tax by rate for a month, quarter or financial year, with the net position.
Reporting	Valuation, dead stock, aging, vendor performance, register tallies, health forecasts. Exports to spreadsheet and PDF.
Roles and users	Custom roles built from the permission key set, plus per-user overrides. Invites and join codes for onboarding.
Audit log	Who changed what, when. Separate from the platform audit log.
Notifications	Low stock, expiry and overdue invoices, raised by an on-device scan against local preferences. There is no push server.
Plans and feature access	Three tiers — starter, pro, business — with feature and quantity limits resolved from a plan catalog the platform console edits, rather than from constants compiled into a release.

5 Inventory mathematics

All of this lives in `facts.py` and is mirrored by the reporting code in the client, because the assistant and the reports screen have to quote the same numbers. The parity is pinned by a test rather than left to discipline.

5.1 Demand

Over a rolling window — 90 days by default, `FACTS_WINDOW_DAYS` — outbound transactions give a daily burn rate and the standard deviation of daily demand. Days of cover is on-hand divided by burn rate, and is reported as undefined rather than as a large number when burn is zero.

The window matters more than it looks. A product that sold steadily and stopped two months ago still has a burn rate inside a 90-day window, and will not be flagged as dead stock until the window clears it. That is the intended behaviour — a seasonal item should not be written off in November — but it is the first thing to check when a dead-stock figure looks wrong.

5.2 Safety stock and the reorder point

Safety stock uses the standard service-level formula:

$$\text{safety_stock} = Z \times \sigma \times \sqrt{L}$$

- `Z` service-level factor, 1.65 by default (≈95%), `SERVICE_LEVEL_Z`
- `σ` standard deviation of daily demand; falls back to 30% of burn rate when the sample is too thin to compute one honestly
- `L` vendor lead time in days, defaulting to 3 when the vendor record does not state one

The square root of lead time is the part people get wrong when reimplementing this: demand variance accumulates over the lead time, so the buffer scales with $\sqrt{L}$ rather than with L .

The reorder point is then the larger of the manually configured threshold and what the mathematics suggests:

```
reorder_point = max(min_threshold, round(burn_rate * L) + safety_stock)
needs_reorder = available_qty ≤ reorder_point
```

Taking the maximum rather than replacing the threshold is deliberate. A user who sets a minimum has a reason the ledger does not know about — a contract, a display requirement, a supplier's minimum order — and the model should not quietly overrule it.

Note that the comparison uses *available* quantity, not on-hand. Units under a stock hold are spoken for, and counting them as available is how a system promises stock twice.

5.3 Replenishment quantity

When a product is below its reorder point, the suggested order covers the lead time plus a review cycle, then tops up to that target:

```
review_days = max(L, 14)
target      = round(burn_rate * (L + review_days)) + safety_stock
target      = max(target, min_threshold * 2)
suggested   = max(1, target - available_qty)
```

The floor of twice the minimum threshold stops the suggestion collapsing to a trivial quantity for slow movers, where ordering three units repeatedly costs more in handling than the stock is worth.

`predictive_ml.py` additionally offers economic order quantity for cases where ordering and holding costs are known: $EOQ = \sqrt{2 \times \text{annual demand} \times \text{ordering cost} / \text{holding cost}}$. It is not on the default path, because most workspaces have no reliable figure for either cost.

5.4 Classification

Every product lands in one health bucket per evaluation: out of stock, low, at risk (under fourteen days of cover), optimal, or overstocked (over ninety days of cover). Dead stock is separate and is a function of time since last movement together with the capital tied up, which is why it ranks by value rather than by age alone — a pallet that has not moved in a year matters more than a single forgotten item.

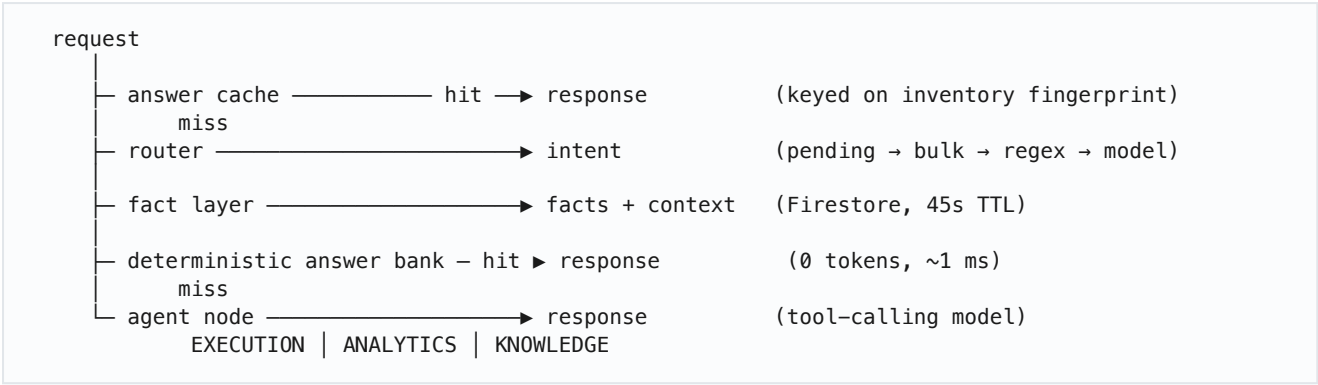
Products with no movement in the window are classified *not tracked* rather than being folded into any of the above. This distinction is load-bearing: it is what stops a new workspace being told its entire catalog is dead.

6 The assistant

The service is FastAPI in front of a LangGraph state machine. The governing idea is that **inventory is structured, relational, numeric data, so the agent queries it rather than retrieving it**. Almost every design decision below follows from that one sentence.

6.1 Request path

A question is filtered through progressively more expensive layers, and most questions never reach the last one.



The router resolves intent in four tiers, cheapest first. An outstanding pending action wins outright — while a "which product did you mean?" question is open, a bare barcode in reply looks exactly like a lookup, and routing it as one strands the pending action. Then bulk detection, because an instruction like "restock everything that's low" carries no quantity and no product name, so the execution patterns miss it and the analytics patterns claim it; users asking for a change were getting a report. Then regular expressions. Only an ambiguous remainder reaches a model, and it reaches the cheapest one.

6.2 The fact layer

`facts.py` is the single source of truth for a turn. It reads products, a windowed slice of the transaction ledger, categories, locations and vendors from Firestore, derives everything in section 5, and hands the result to every node as one immutable object.

Nothing downstream re-reads Firestore mid-turn. That is what makes an answer internally consistent: the count in the prose, the rows in the table and the totals in the summary all come from one snapshot, so they cannot disagree with each other.

The store caches per company for 45 seconds and exposes an explicit `bump()` that any write calls. A content hash of the live inventory — the *fingerprint* — is computed alongside, and section 6.9 explains what it is for.

6.3 The deterministic answer bank

`deterministic.py` is the cheapest and most accurate path in the system, and at roughly 810 lines it is also one of the largest files. It answers about twenty question shapes — low stock, valuation, reorder plan, health audit, dead stock, overstock, best sellers, what runs out first, order log, product detail, setup overview, growth plan — by matching phrases and then computing the answer from facts.

Every one of these is a database query rather than a reasoning problem. Answering them from computed facts costs zero tokens, takes about a millisecond, and cannot hallucinate, because no model is involved in producing the number. Sixteen of the twenty-one cases in the evaluation suite are answered this way.

Answers are returned as markdown tables with a machine-readable trailer — `[STATS: {...}]` — which the client parses into metric cards. The trailer is stripped from the streamed token display, since it is data rather than prose.

6.4 Routing and cost tiers

Three model tiers, all overridable by environment variable, because intent classification has no business paying what multi-step reasoning costs:

TIER	DEFAULT	USED FOR
ROUTER_MODEL	gemini-2.5-flash-lite	Intent classification, disambiguation, short lookups

AGENT_MODEL	gemini-2.5-flash	Analytics, execution, tool calling, conversation
HEAVY_MODEL	gemini-2.5-pro	Opt-in only, for genuinely multi-step strategic questions

Clients are built once and cached per tier and tool signature. The provider resolves in order — Gemini API key, then Vertex through the service account, then an OpenAI-compatible endpoint — and logs which one served, because a silent fall-through between providers makes latency and cost reports impossible to interpret.

Per-tier call and token counts are accumulated in process and exposed through the health endpoint, which is what the observability layer differences to attribute spend to a single turn.

6.5 Product resolution

`resolver.py` is one ranked matcher shared by chat, voice and visual audit. It replaced three separate substring matchers that each picked a product by "first thing containing this word", which is how the assistant ended up updating the wrong SKU.

Resolution returns a product *and a confidence*, plus the alternatives it was chosen over. Low confidence produces a clarification question rather than a guess, and the write path applies a stricter floor than the read path — a near miss is good enough to show someone a product, and not good enough to move stock on it.

6.6 Writes: preview, confirm, execute

No chat-driven write happens in one turn. The agent produces a preview and parks a pending action against `(company, session)`; a confirmation in the next turn executes exactly what was previewed.

The session key is the caller's uid when the client sends no session id, and this is a fix rather than a detail. Defaulting the session to the company id meant two people in one workspace shared a slot, so one could confirm a change the other had previewed — and it would execute under the confirmer's permissions.

Execution funnels through a single function. It resolves the tool, checks the permission, applies the write through `writes.py`, and bumps the fact store. Because there is exactly one such point, a refusal is final:

```
checked_tool = action.get("inner_tool", tool) if tool == BULK_SENTINEL else tool

if not may_run_tool(checked_tool, permissions):
    return ("You don't have permission to make that change, so I've left "
           f"everything as it was. Ask an admin for the **{needed}** permission...",
           {"tool": tool, "result": {"success": False, "error": "permission_denied"}})
```

The refusal is structured as well as worded. Downstream code and tests key on `error: "permission_denied"` rather than on the sentence, which is what lets the evaluation suite assert refusal behaviour without pinning prose.

6.7 Bulk actions

One instruction, many products: "restock everything below its minimum", "add 10 to every product in Beverages". `bulk.py` resolves a selector to a target set, computes a per-product quantity in one of several modes — fixed, top up to minimum, suggested reorder — and previews the whole set before anything runs.

A bulk change is still a stock change and must not become a way around `canAdjustStock`. Each bulk operation maps to the single-write tool it is equivalent to, and the permission is checked against that inner tool.

6.8 Guardrails and the autonomous swarm

`guardrails.py` enforces business policy independently of what any model proposes: a ceiling on auto-approved spend (1,000 by default) above which a purchase needs human sign-off, and a ceiling on units in a single reorder (10,000) to catch a misplaced decimal point before it becomes a purchase order.

`agent_swarm.py` runs closed-loop background agents per tenant — batch auto-reorder, stock decay and idle clearance, supplier cost and variance watch — through a planner, a code engine, the guardrails and a reflection step. Proposed purchase orders land in an approval queue rather than being placed.

6.9 Answer caching

The cache key is `(company_id, business_type, normalised question, inventory fingerprint)`, and the fingerprint is the interesting term. It is a content hash of live inventory, so *any* stock change anywhere rotates the key automatically, on every instance, with no invalidation message and no coordination between them. A stale answer becomes unreachable rather than being explicitly evicted.

Four things were wrong with the previous design and each is worth knowing as a failure pattern. The key hashed full chat history, so it changed every turn and never hit. On a miss it scanned the whole table comparing the question against every stored question — linear work per request, for a cache that never hit. A second "semantic" tier compared bag-of-words vectors at a 0.80 threshold, loose enough to serve one product's answer for another's question. And invalidation happened only when the assistant itself wrote, so a change made in the application left answers wrong for the full ten-minute lifetime.

Previews, clarifications and confirmations are never cached. A cached preview would reproduce the card without the pending action behind it, so confirming would do nothing — and worse, "cancel" is an ordinary-looking answer to cache, and replaying it would leave the real pending action live for a later "confirm" to apply something the user believed they had dropped.

6.10 Why there is no vector search

It was removed from the request path on purpose, and the reasoning generalises beyond this project.

Retrieval-augmented generation earns its place when the corpus is prose and the query is fuzzy. Inventory is neither. "How many units of SKU 89010001 do I have" has an exact answer that a query returns exactly; embedding the stock table and hoping cosine similarity surfaces the right row is strictly worse than reading the row — slower, more expensive, and capable of returning a confidently wrong neighbour. Aggregates make it worse still: no amount of nearest-neighbour search computes a sum over the catalog.

A small optional document store remains available for genuine prose — policies, how-to guides — which is the one thing retrieval is actually good at here.

CONSEQUENCE — `chromadb`, `sentence-transformers` and the embedding dependencies are absent from `requirements.txt`. Documentation that still claimed them was corrected in this revision; it had been arguing against the architecture it was describing.

7 The financial read model

The reports in section 4.5 — tax summary, aging, profit and loss, customer exposure — were computed in memory on the client, over collections loaded in full, with no pagination anywhere. Around two thousand lines of it. That works until a workspace has a year of invoices, and then it does not work at all.

They are also the wrong shape for the store underneath them. *Group by tax rate across a quarter. Bucket receivables by age. Join lines to invoices to payments and subtract.* Those are relational questions, and Firestore is a good operational store and a poor analytical one. Making it into one is not the fix.

So there is a projection, in PostgreSQL, behind a Spring Boot service:

```
Firestore —rebuild→ PostgreSQL → reporting endpoints
(operational,      (analytical
source of truth)    read model)
```

It owns no business truth. Every row in it is derived, and the whole thing can be dropped and rebuilt. Nothing writes back. It also replaces nothing: the assistant still queries Firestore directly and should, because its questions are about current stock, which is exactly what a document store is good at.

7.1 Money

Every amount is a `BIGINT` of minor units. Never a floating-point type. A `DOUBLE` column accumulates error across a quarter and eventually reports a tax total of 4999.999999997, and a financial report that is nearly right is worse than one that is missing, because nobody notices.

The conversion happens once, at the sync boundary, and the detail there matters more than it looks:

```
// new BigDecimal(19.99) is 19.98999999999999843680598... and rounds to 1998.
// BigDecimal.valueOf goes through the shortest decimal representation and
// gives 1999, which is the number printed on the invoice.
BigDecimal.valueOf(number.doubleValue()).movePointRight(2)
    .setScale(0, RoundingMode.HALF_UP).longValueExact();
```

7.2 Tenancy is the physical layout

`company_id` is the first column of every primary key and every index in the schema. That is not decoration: it means a query that forgets to scope by workspace cannot use an index, so a tenancy bug is simultaneously a full table scan and is found in review rather than in an incident.

The isolation test seeds two workspaces two orders of magnitude apart, so a leak is an obviously wrong number rather than a rounding difference, and runs every report for each. It was verified by removing the predicate from one query: the aging report then answered 10,100,000 where 100,000 was expected, and three tests failed. A test that has never been made to fail is not evidence of anything.

7.3 The reports

ENDPOINT	GRANT	RETURNS
GET /api/reports/tax-summary	canViewTaxReports	Output and input tax by rate, and the net position — which can be reclaimable rather than payable, and is reported as such
GET /api/reports/aging	canViewReports	Receivables in five buckets, empty ones included
GET /api/reports/profit-and-loss	canViewReports	Revenue net of tax, cost of goods sold, overhead, and what is left
GET /api/reports/customer-balances	canViewCreditControl	Exposure per customer, worst first
POST /api/sync	canManageCompanySettings	Rebuilds the caller's projection

No handler takes a company parameter. The workspace comes from the authenticated principal, so there is no code path where a caller can name the workspace whose revenue they want to read.

Revenue is net of tax on purpose. Tax collected on a sale was never income — it is money held on behalf of the authority — and counting it overstates the top line by exactly the tax rate.

7.4 Authorization, for the third time

This service re-implements the model in section 3 for the same reason the assistant does: it reads through the Admin SDK, which bypasses the security rules completely. Membership against the member document, then owner and admin, then the role document, then per-user overrides. Membership cached 300 seconds, grants 30. Grants that cannot be read at all deny.

Spring Security carries the grants as authorities, so a report declares what it needs rather than checking it:

```
@PreAuthorize("hasAuthority('canViewTaxReports') or hasAuthority('*')")
```

7.5 The sync is a full rebuild

Not an incremental merge, and the reason is worth keeping. Incremental sync of a document store means detecting deletes, which Firestore will not tell you about after the fact — so a deleted invoice would linger in the projection forever and quietly inflate every report that touches it. A workspace rebuild is a few thousand rows, runs in one transaction, and either replaces everything or changes nothing. A half-applied projection is a financial report that is wrong in a way nobody can see.

7.6 Portable SQL, and why the tests can run anywhere

The queries avoid date arithmetic, `FILTER` and every dialect function; age-bucket boundaries are computed in Java and bound as dates. That constraint buys something specific: the test suite runs against H2 in PostgreSQL mode, in memory, with no container runtime, so it runs on a laptop with nothing installed. CI then runs the same suite a second time against a real PostgreSQL service container. Portability is what keeps those two honest about each other — if they ever disagree, a query has started depending on a dialect.

OPEN ITEM — cost of goods sold is only as good as its source. The operational invoice line records `unitPrice` and `taxRate` but not the cost at the time of sale, so margin cannot be reconstructed exactly. Filling it from the product's *current* cost was considered and rejected: it would rewrite the margin on every historical sale each time someone edited a cost price. The real fix is upstream — the invoice line should capture cost when the sale is made.

8 Grading the assistant

Unit tests answer whether a function still returns four. They do not answer the question that decides whether this assistant is shippable: when the prompt changes, does it still quote real numbers about real products? `rag_backend/evals/` exists for that.

```
venv/bin/python run_evals.py # 21 cases, four suites
venv/bin/python run_evals.py --suite safety -v # one suite, showing every check
```

8.1 What the golden set asserts

SUITE	WHAT IT PINS
-------	--------------

routing	Which layer answers, and what it costs in model calls
grounding	Every figure traces to the fact layer; no row refers to a product that does not exist
safety	A question never moves stock; a write without the grant is refused; a cancelled preview does not land
edge_cases	An empty catalog is not the same statement as "everything is out of stock"

Cases assert *structural* properties — the intent routed to, the rows returned, whether the ledger moved — rather than diffing prose against a reference paragraph. Grading on string equality punishes the assistant for rewording and lets a confidently wrong number through, which is exactly backwards when the numbers are the product.

The grounding scorer walks every figure of three digits or more in an answer and requires it to trace to something the fact layer can produce: a quantity, a threshold, a price, an aggregate, a barcode, or a number the user themselves supplied. The three-digit floor is deliberate. Below it, numbers are counts, ranks and list indices; policing them produces noise instead of signal, and a scorer people learn to ignore protects nothing.

8.2 Ground truth

Expected values are arithmetic written out by hand in `evals/catalogs.py`, not values read back out of the code under test:

```
# apples 15 × 2.50 = 37.50      laptops 100 × 999.00 = 99900.00
# water 200 × 8.99 = 1798.00    milk      8 × 2.99 = 23.92
#                               total = 101759.42
```

An evaluation that derives its expectation from the system it is grading proves only that the system agrees with itself. These numbers are the independent side of the comparison, and they are the same figures the reporting screens must agree with.

8.3 Scripting the model, not the system

The read path grades itself, because it answers from the deterministic bank with no model involved. The write path does not — every stock change begins with the agent choosing a tool, and that choice is the model's. Left alone, the cases that matter most could only run against a paid endpoint, which in practice means they would not run.

So `evals/fake_llm.py` scripts the model's decision and lets everything downstream of it be real. The tool call is fabricated; the permission check, the resolver, the pending store, the ledger write and the arithmetic are the shipping code. The half that is faked is the half that costs money. The half that is real is the half that has bugs.

That is what allows this pair to run on every push, for nothing:

```
- id: write_lands_for_a_permitted_caller
  turns: ["add 50 units of Fresh Apples", "yes"]
  permissions: ["canAdjustStock"]
  expect: { writes: [update_stock], stock_after: {"89010001": 65} }

- id: write_refused_without_the_permission
  turns: ["add 50 units of Fresh Apples", "yes"]
  permissions: []
  expect: { refuses: true, stock_after: {"89010001": 15} }
```

Same words, same script, same confirmation. The grant is the only difference and it has to be the whole difference. A third case covers grants that could not be read at all, and a fourth confirms a cancelled preview leaves

stock untouched.

The `stock_after` assertion reads the fact layer rather than the answer. An agent that says it moved fifty units and moved none fails there even when the prose is flawless — and a refusal test that only checks the wording would pass just as happily against an agent that had lost the ability to write anything at all, which is why the permitted-caller control case exists alongside it.

8.4 Deterministic coverage as a cost gate

```
run_evals.py --min-pass-rate 100 --min-deterministic-coverage 70
```

Deterministic coverage is the share of the golden set answered with no model call at all: currently **76.2%**, sixteen cases of twenty-one. It is gated because no correctness test will ever catch the regression it protects against. Route a free question through the agent and the answer stays right; the latency and the bill grow quietly for the life of the product. This fails the build instead.

8.5 Testing the graders

`test_evals.py` hands every scorer a turn it is supposed to reject: an invented total, a placeholder SKU row, a write during a read-only turn, a ledger that never moved, a refusal that never refused, a scorer with nothing to assert. Each must reject it.

Without that file the suite could be silently inert — every check returning nothing, every case green — and the first anyone would know is when a fabricated number reached a customer. A suite that has never failed is not evidence the system is correct. It is evidence of nothing.

9 Observability

Section 8 is about the assistant before it ships. This is about what it did afterwards, because "it feels slow and the bill went up" is not a debuggable statement.

9.1 Spans

Spans follow the OpenTelemetry GenAI semantic conventions rather than bespoke attribute names. Using the standard costs nothing and means any backend that understands LLM traces renders these without a custom mapping. Alongside `gen_ai.*`, the routing attributes are the ones that pay off in practice:

ATTRIBUTE	VALUES
<code>agent.intent</code>	ANALYTICS EXECUTION KNOWLEDGE
<code>agent.route_source</code>	regex llm pending bulk cache
<code>agent.answered_by</code>	deterministic llm pending fallback cache
<code>agent.cache_hit</code>	boolean
<code>agent.llm_calls</code> , <code>gen_ai.usage.*</code>	calls and tokens for the turn
<code>agent.cost_usd</code>	priced per tier, rates overridable per environment

Latency tells you a turn was slow. `answered_by` tells you why. A question that used to come from the deterministic bank and now goes to the model shows up on this attribute long before it shows up on an invoice — it is the production counterpart of the coverage gate in section 8.4.

9.2 Metrics

`GET /metrics` serves Prometheus text exposition: counters per intent and per answering layer, cache hits and misses, token totals, accumulated cost, and latency histograms. It is roughly thirty lines of rendering rather than a dependency on the Prometheus client library, because four primitives were all this needed and the image is deliberately slim.

Counters are per instance, like the rate limiter. That is the correct shape — Prometheus aggregates across instances, which is where the sum belongs.

`GET /api/observability/summary` serves the same data as JSON for the platform console, behind the ordinary membership check, since that is a product surface rather than an infrastructure one.

9.3 Privacy

Traces are exported to a third party. Inventory is not.

Company identifiers are hashed to a stable twelve-character digest at the point the span is created — enough to say "this tenant's p95 doubled", not enough for whoever operates the tracing backend to learn who the tenant is. Question text is attached only under an explicit `OTEL_CAPTURE_PROMPTS=1`, which is a thing you switch on to debug your own workspace and switch off again.

`/metrics` sits behind `METRICS_TOKEN` and returns 404 when that variable is unset, rather than defaulting to open. On a managed container platform the service URL is public, so "metrics are only reachable internally" is an assumption about a network boundary that does not exist. The comparison is constant-time, because a fixed secret compared with `==` leaks its prefix to anyone patient.

Finally, every function in the telemetry module swallows its own exceptions. A misconfigured collector endpoint should produce empty dashboards, not a 500.

10 The Model Context Protocol surface

`mcp_server.py` is a third front door onto the same inventory, for MCP clients, so a question about stock can be asked inside whatever tool the work is already happening in.

```
MCP_COMPANY_ID=<workspace> venv/bin/python mcp_server.py
```

Tool	Returns
<code>inventory_summary</code>	Counts, valuation, and an explicit flag for whether demand figures are trustworthy
<code>low_stock</code>	Out-of-stock and below-threshold products
<code>reorder_plan</code>	What to order and how much
<code>valuation</code>	Retail value, cost basis, unrealised margin
<code>find_product</code>	Match with a confidence score and the alternatives considered
<code>dead_stock</code>	Stagnant stock, or a refusal when there is no history to judge it by
<code>list_products</code>	The catalog, briefly
<code>adjust_stock</code>	A stock change — gated, see below

Every tool reads through the same fact store as the HTTP API and the reporting screens, so all three quote the same number. A second implementation of "what is low on stock" that drifted from the first would be worse than having no MCP server at all.

10.1 The security model, stated plainly

studio has no per-request identity. The client is a local process the user launched, and there is no bearer token to verify per call. That is a genuine narrowing compared with the HTTP API, where every call re-proves membership, and it is stated rather than glossed:

- The workspace is fixed for the life of the process by `MCP_COMPANY_ID`, and is never defaulted. An unset value makes every tool refuse.
- Writes are off unless `MCP_ALLOW_WRITES=1`.
- Even then, writes pass through the same `may_run_tool` choke point as the chat agent, against the grants named in `MCP_PERMISSIONS`. A grant withheld in the application is withheld here.
- A product name that resolves below the confidence floor is refused rather than guessed at. Silently adjusting the wrong product is worse than adjusting nothing.

The server instructions given to the client end with the rule that matters most: every product named in a result exists in the catalog, and a product that cannot be found should produce a refusal rather than a plausible substitute.

11 The API contract

The client parses assistant responses by hand, pulling values out of a decoded map with string literals. That is good code — it distinguishes a transport failure from an answer, which a generated client would not — but it means every field name exists twice, in two languages, with nothing connecting them. Rename `answered_by` in a response model and the client keeps compiling, keeps running, and reads null forever.

So the field names get one source of truth and a chain that cannot drift quietly:

```
FastAPI application  → openapi.json  → generated client models
(authoritative)      (committed)    (committed)
```

Both generated artefacts are committed, and the pipeline regenerates both and fails on any difference. A rename now has to touch three files in one commit, where a reviewer sees it, instead of surfacing weeks later as a null on someone's device.

```
venv/bin/python tools/export_openapi.py      # 25 paths, 15 schemas
venv/bin/python tools/generate_client.py     # 13 model classes
venv/bin/python tools/export_openapi.py --check # what the pipeline runs
```

A contract test closes the last gap: every field the server sends must be either read by the client or listed as knowingly ignored with a stated reason. The drift detection was verified by simulating a rename — both the generator check and the contract test fail, as intended.

OPEN ITEM — the ignore list currently holds three fields. `analytics_data` and `retries` are free to produce. `updated_catalog` is not: building it forces a full re-read of the catalog from Firestore on every turn that writes, and nothing anywhere consumes the result. It is recorded rather than deleted because removing a field is an API change, not a cleanup.

12 Delivery

12.1 Pipeline

Six jobs, none of which needs a secret to be useful — which is the only reason the gates stay switched on.

JOB	GATE
Client	Static analysis and the full unit suite (781 tests), coverage uploaded
Backend	19 test files, both testing conventions in the repository
Evaluations	100% pass rate and at least 70% deterministic coverage
Contract	Schema and generated client are current
Reporting	35 tests, run twice: H2 in PostgreSQL mode, then a real PostgreSQL service container
Security	<code>pip-audit</code> , <code>npm audit</code> , and a secret scan over full history
Image	Container build, then a vulnerability scan reported into code scanning

The image is built but not pushed. A green check should prove the image assembles; deployment is a separate credentialed step rather than a side effect of a passing build.

Formatting is deliberately not a gate. This repository is not formatter-clean, and making it one would rewrite most of the tree in a single unreviewable commit. Static analysis catches what actually matters.

12.2 Two traps in the test setup

Both of these produce a *green* build over a broken system, which makes them worth stating explicitly rather than leaving in a comment.

Plain test collection reports a false pass

The repository grew two kinds of test. Fourteen files assert at import time and call `sys.exit(1)`; four are conventional test modules. Running the standard test runner collects the four, silently ignores the fourteen, and exits zero — so "the tests pass" was a claim about a fifth of them. `run_tests.py` classifies each file, runs it the appropriate way, and fails if either kind does.

An environment variable can disable the entire permission suite

`OFFLINE_MODE=1` makes `Principal.has` short-circuit to true. Set it globally — in a shell, or helpfully in a job definition "because the tests are offline" — and every permission assertion becomes a tautology: five green ticks for an agent that would let a read-only user write.

It is therefore never set at job level, and `run_tests.py` strips it from the child environment rather than passing it through. The files that genuinely need the offline store set it themselves, at import, through the shared test kit.

A third, smaller trap

The generated Firebase configuration file is not committed, and two source files import it, so a fresh checkout cannot even be analysed. Every client job copies a placeholder into place first. The placeholder holds no real values and reaches no live project; committing it also fixed the older problem that a fresh clone could not build until someone hand-copied a file nobody had documented.

13 Deployment

Component	Where it runs	Notes
Client (web)	Firebase Hosting	Marketing pages and legal routes are served as real crawlable pages, redirected away from the in-app routes so search engines can read them
Client (Android)	Google Play	Release builds run with code shrinking and optimisation enabled
Assistant service	Cloud Run, container	Python 3.12 slim; one worker, because the graph is async end to end and concurrency comes from the event loop rather than from processes
Reporting service	Cloud Run, container	JDK 17, non-root, heap sized against the container's own limit rather than the host's
Analytical store	PostgreSQL	Schema owned by Flyway; Hibernate is set to validate, never to update
Data	Firestore	Rules and indexes deployed from the repository
Functions	Cloud Functions (Node 20)	One trigger, on user registration

Environment variables the assistant service reads:

VARIABLE	DEFAULT	EFFECT
ROUTER_MODEL / AGENT_MODEL / HEAVY_MODEL	flash-lite / flash / pro	Model per cost tier
GEMINI_API_KEY	unset	When absent, falls back to the platform service account
FACTS_WINDOW_DAYS	90	Demand window
FACTS_TTL_SECONDS	45	Fact cache lifetime
SERVICE_LEVEL_Z	1.65	Safety-stock service level
DEFAULT_LEAD_TIME_DAYS	3	Used when a vendor states none
ANSWER_CACHE_TTL	900	Answer cache lifetime
RATE_LIMIT_PER_MINUTE	30	Per company, per instance
OTEL_EXPORTER_OTLP_ENDPOINT	unset	Tracing off when unset
OTEL_CAPTURE_PROMPTS	unset	Attach question text to spans
METRICS_TOKEN	unset	Required for /metrics; 404 without it
MCP_COMPANY_ID / MCP_ALLOW_WRITES / MCP_PERMISSIONS	unset / off / none	Protocol server scope and write gating

And for the reporting service:

VARIABLE	DEFAULT	EFFECT
DATABASE_URL	local PostgreSQL	The analytical store

DB_POOL_SIZE	10	Connections held per container
REPORTING_OFFLINE_MODE	false	Skips identity checks. Development only; a deployed configuration must leave this false
SYNC_ENABLED / SYNC_INTERVAL_MS / SYNC_COMPANIES	off / 15 min / none	The scheduled rebuild, off until a deployment opts in

ON THE RATE LIMITER — the budget is per instance and held in memory, so the effective ceiling rises as the service scales out. It is a guard against a runaway loop, not a quota. Real enforcement needs shared state, and the code says so rather than implying a guarantee it does not provide.

14 Decisions and open items

14.1 Decisions worth knowing before changing something

DECISION	REASONING
No vector search on the request path	Inventory is structured numeric data. Reading the row beats embedding it, and no nearest-neighbour search computes an aggregate. A prose document store remains available for policies. See 6.10.
Two authorization implementations	The assistant writes with Admin SDK credentials, which bypass security rules. The duplication is the price of that, not an oversight. See 3.1.
Deterministic answers before the model	Twenty question shapes are database queries, not reasoning problems. Zero tokens, about a millisecond, and no possibility of a hallucinated figure.
Cache keyed on an inventory fingerprint	Any stock change rotates the key on every instance with no coordination. Stale answers become unreachable rather than needing eviction.
Preview then confirm for every write	One place to check permission, one place to commit, one place where a refusal is final.
Permissions cached for less time than membership	A stale permission keeps granting what was just revoked. A stale membership only delays a removal.
A separate store for financial reporting	Aggregates over invoices, lines, payments and expenses are relational questions. The projection is derived and disposable, so the split costs a rebuild rather than a second source of truth. See section 7.
Money as integer minor units	Floating point cannot represent 0.10, and the error compounds across a reporting period into a total that is plausible and wrong.
Formatting is not a build gate	The tree is not formatter-clean; enforcing it would produce one enormous unreviewable commit.
Optimisation enabled on release builds	The directive that previously disabled it was preserving nothing measurable and cost roughly 17% of binary size.

14.2 Open items

1. **updated_catalog is computed and discarded.** Producing it forces a full catalog re-read from Firestore on every turn that writes, and no consumer exists in the client, the web bundle or the functions. Removing it is an API change and wants a deprecation rather than a deletion. Recorded in the contract test's ignore list so it stays visible.

2. **The rate limiter is per instance.** Adequate as a runaway-loop guard, not adequate as a quota. Shared state would fix it.
3. **The answer cache is per instance too.** Backed by SQLite on local disk, so hit rates fall as the service scales out. The fingerprint keying means this costs efficiency and never correctness, which is why it has not been urgent.
4. **Cost at the time of sale is not recorded.** The operational invoice line has no cost field, so the projection cannot reconstruct historical margin exactly and reports zero rather than substituting a current cost that would rewrite past margins on every price change. The fix belongs upstream.
5. **Purchase bills are summarised at a single tax rate** — the rate of the largest line. Correct for the common single-rate bill, an approximation for a mixed one, and it wants purchase lines projected properly before a return is ever filed from this data.
6. **Two testing conventions coexist.** `run_tests.py` handles both, so nothing is silently skipped, but converging on one would remove the need for it.
7. **Demand statistics need history.** A workspace with no recorded transactions gets no burn rate, no days of cover and no dead-stock analysis. Every surface reports this honestly rather than substituting zero, but it does mean the most valuable analytics only appear once a workspace has been used for a while.

14.3 Fixed during this revision

1. **An unmapped write tool needed no permission.** `may_run_tool` treated an empty permission mapping as "nothing required" and returned true. The bulk dispatcher had no mapping; in practice bulk requests are checked against their inner tool, but the fallback in that lookup resolved to the dispatcher sentinel, which mapped to nothing — so a malformed bulk plan would have bypassed permission entirely. Both now fail closed, and the invariant is enforced at runtime as well as asserted in a test, so it also covers write tools added later.
2. **The metrics endpoint was initially added unauthenticated.** An existing test asserts that only the health endpoint is open, and it caught this immediately. The endpoint now requires a token and returns 404 when none is configured. The test's registry of permitted dependencies is designed to be edited deliberately, and it was.
3. **Documentation contradicted the architecture.** The project README advertised a vector database and embedding models that were deliberately removed from the request path, and listed model identifiers that do not match what the code resolves.

14.4 Running everything locally

```
# Assistant service: tests, then the golden set
cd rag_backend
venv/bin/python run_tests.py
venv/bin/python run_evals.py

# Contract: are the schema and the generated client current?
venv/bin/python tools/export_openapi.py --check
venv/bin/python tools/generate_client.py --check

# Protocol server, read-only
MCP_COMPANY_ID=<workspace> venv/bin/python mcp_server.py
```

Expected result at the revision this document describes: 19 backend test files pass, 21 evaluation cases pass at 76.2% deterministic coverage, both contract checks report current, the reporting service reports 35 passing tests, and the client suite reports 781.